AutoSystemDesign

От бизнес-требований к проверяемой модели архитектуры микросервисной системы

Дичковский Владимир

Какую задачу решает инструмент

Проблема

- В BRD описаны функции и ограничения, а не компоненты.
- Архитектор вручную переводит требования в схемы.
- Согласованность диаграмм компонентов, сценариев и API приходится проверять.

Решение

- LLM предлагает структурированную модель.
- Доменный код проверяет её правилами.
- Из одной модели строятся связанные артефакты.

Система помогает визуализировать архитектурные гипотезы для ревью и дальнейшего обсуждения.

Вход: BRD с бизнес-требованиями

Демонстрационный фрагмент BRD интернет-магазина

Функциональные требования

- Корзина и подтверждение заказа.
- Проверка остатков и расчёт цены.
- Оплата, создание заказа, уведомление.
- Просмотр статуса заказа покупателем.

Нефункциональные требования

- Обычно: 120 запросов/с
- В пике: 400 запросов/с
- p95 не выше 300 мс
- Доступность: 99,9%

BRD — исходные бизнес-формулировки. Состав сервисов пока не задан.

Результат извлечения: требования

Наглядный пример структурирования BRD

Функциональные

- FR-01: оформление заказа
- FR-02: проверка наличия товаров
- FR-03: оплата и подтверждение
- FR-04: просмотр статуса

Нефункциональные

- Нагрузка: 120 / 400 RPS
- Целевое p95: 300 мс
- Доступность: 99,9%

След происхождения: для каждого значения сохраняется цитата BRD; конфликтующие формулировки фиксируются отдельно.

Извлечение не означает безусловного доверия LLM

Двухэтапная проверка

1. LLM выделяет требование и цитату.
2. Код сверяет цитату с BRD.
3. LLM сопоставляет подтверждённое требование с полями модели.
4. Значения ещё раз проверяются по источнику.

Сомнительные данные

- Неподтверждённые числа исключаются.
- Противоречия сохраняются для человека.
- Отсутствующие данные не придумываются.

Пример неоднозначности BRD: доставка письма за 10 секунд или до 2 минут в пике? Решение нужно уточнить.

Архитектура конвейера и зависимости этапов



Performance зависит от C4; API опирается на C4 и Sequence.

Как LLM превращается в проверяемый генератор

Генерация

- LiteLLM: единая точка работы с моделями.
- LLM возвращает структурированный ответ.
- Pydantic преобразует его в типизированную доменную модель.
- Код PlantUML строит отдельный рендерер, не сама LLM.

Если нарушены правила

- Ошибки разбора и доменной проверки возвращаются модели.
- Модель исправляет ответ.
- Число попыток ограничено.

Принцип: генеративная модель предлагает, детерминированные валидаторы принимают или отклоняют.

C4: не картинка, а модель предметной области

Типы компонентов

- Клиент, балансировщик
- Вычислительный сервис
- База данных и кэш
- Брокер сообщений

Параметры модели

- Реплики и лимит параллелизма
- Время обработки и разброс
- Размер очереди и стоимость
- Протокол и тип маршрутизации связи

Зачем: одна и та же модель нужна и для рисования диаграммы, и для расчёта нагрузки и правил валидации.

Валидаторы C4: три уровня проверки

Структура

Уникальные ID, допустимые связи, отсутствие циклов

Семантика

Поставщик и потребитель брокера, путь мимо кэша

Расчёты

Поток RPS, пропускная способность, насыщение

Допустимо

Клиент → API → БД

Недопустимо

Клиент → БД напрямую

Онтология допустимых связей ограничивает пространство решений, которые может предложить LLM.

Пример: диаграмма компонентов C4

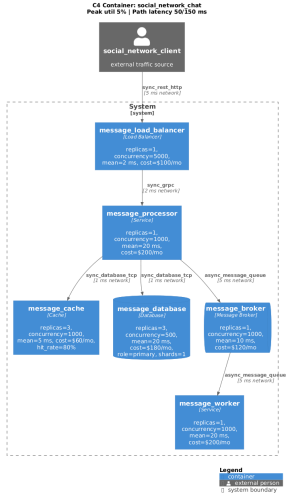


Иллюстрация из репозитория. Позже будет заменена результатом единого BRD-прогона.

Диаграмма Sequence: от требования к сценарию

Построение

- Сценарий взаимодействия на каждое функциональное требование.
- Участники ссылаются на компоненты диаграммы компонентов.

Проверка

- Все участники есть в модели компонентов.
- Сообщения идут по существующим связям между компонентами.
- Клиент не обращается к базе данных напрямую.

Последовательность объясняет поведение архитектуры, которое не видно на статической диаграмме компонентов.

Пример: диаграмма последовательностей

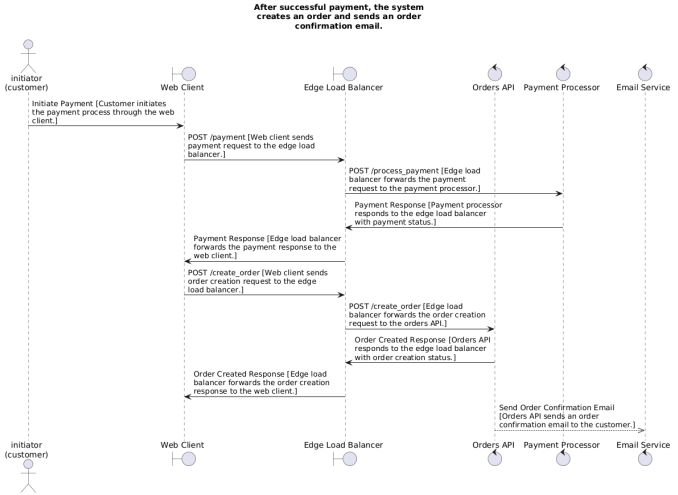


Иллюстрация из репозитория; связь с выбранным BRD ещё будет проверена на едином прогоне.

REST API: контракт и пример OpenAPI JSON

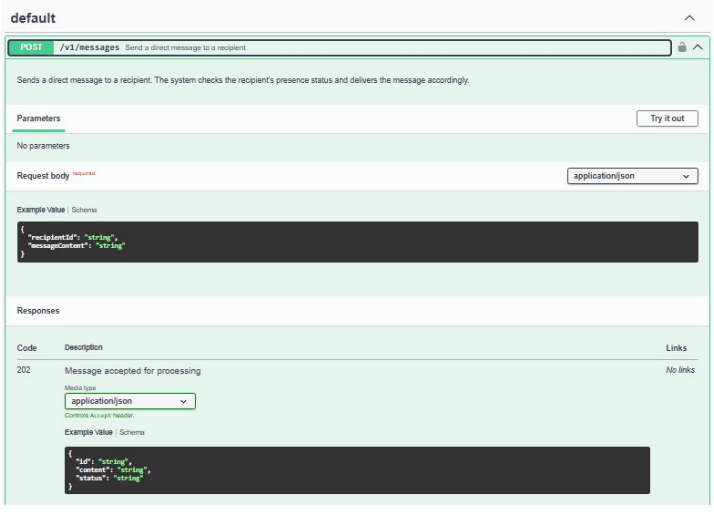
Что получаем

- Методы и маршруты по сценариям.
- Структуры запросов и ответов.
- Проверка уникальности операций и отсутствия конфликтующих маршрутов.

Фрагмент OpenAPI JSON

```
{
  "openapi": "3.1.0",
  "paths": {
    "/orders": {
      "post": {
        "operationId": "createOrder",
        "responses": {
          "201": {"description": "Created"},
          "409": {"description": "Out of stock"}
        }
      }
    }
  }
}
```

Пример: Swagger UI



Существующий скриншот. Для финальной демонстрации нужен Swagger от того же BRD.

Анализ производительности: что считаем

Математическая часть

- Распределение RPS по путям графа.
- Пропускная способность и загрузка узлов.
- Оценка ожидания в очередях и средних задержек.

Условия применимости

- Валидация на основе теории массового обслуживания.
- Число обслуживающих каналов не равно автоматически лимиту параллелизма.
- Неизвестная величина остаётся неизвестной.

Это аналитическая оценка модели, не нагрузочный тест и не моделирование реального кластера.

Пример: отчёт анализа производительности

Анализ производительности

Сценарий: checkout_baseline

Синхронный путь с наибольшей задержкой

Максимальная оценка средней задержки пути: 32.10 мс

Критический путь: client → checkout_api → payment_service

Показатели компонентов

Компонент	Нагрузка запр./с	Пропускная запр./с	Загрузка %	Ожидание мс	Ответ мс
checkout_api	60.00	500.00	12.0	0.02	8.02
inventory_db	60.00	333.33	18.0	0.09	12.09
payment_service	60.00	150.00	40.0	2.08	22.08

Показатели рассчитаны по параметрам сценария, а не измерены на работающей системе.

Пример

60 RPS

нагрузка узлов

32,10 мс

максимальная оценка средней задержки пути

Реальный отчёт checkout_baseline; исходный BRD демонстрационного примера отличается.

Следующие задачи

- Удобный интерфейс загрузки BRD и просмотра результатов.
- Проверка расчётных оценок с помощью SimPy: дискретно-событийное моделирование очередей и сравнение с результатами теории массового обслуживания.

Автоматическая генерация — начало проектирования. Окончательное решение остаётся за архитектором.